



Øving 5: Rotfaste trestrukturer

Kollapser kontekst

←

→

Du må oppnå minst 12 av totalt 31 mulige poeng for å denne øvingen godkjent. Hver flervalgsoppgave er verdt 1 poeng, mens hver kodeoppgave er verdt 3 poeng. For informasjon om øvingsopplegget, se nettsidene. Trenger du hjelp, så ta kontakt.

Løsningsforslaget publiseres på Canvas etter innleveringsfristen.

Spørsmål 1: Refleksjon

Dette er en refleksjonsoppgave. Denne oppgaven teller ikke mot antall oppgaver som må løses for å få godkjent øvingen. For mer informasjon om refleksjonsoppgavene, se nettsidene.

Refleksjonsoppgavene handler kun om refleksjon rundt forståelse av og arbeid med fagstoffet og øvingen. Pass på så du ikke inkluderer noen sensitive personopplysninger i refleksjonen din.

Før du begynner å gjøre øvingen, reflekter over følgende:

- Hvilke deler av pensum knyttet til denne øvingen er du usikker på?
- Hva tror du at du kommer til å lære ved å gjøre denne øvingen?

Etter du er ferdig med øvingen, reflekter over følgende:

- Hva forstår du nå, som du ikke forsto før du begynte å jobbe med øvingen?
- Hvilke nye sammenhenger ser du?
- Kan du komme på flere anvendelser av stoffet fra denne øvingen? I så fall hvilke?
- Er det noe du fortsatt er usikker på? I så fall hva?
- Hvis du skulle gjort øvingen på nytt, hva ville du gjort annerledes?

Spørsmål 2: Indeks - foreldrenode

I en binærhaug har du en node med indeks 9. Hva er indeksen til foreldrenoden?

- ☐ 5
- ☐ 2
- ☒ 4
- ☐ 6
- ☐ 3

Spørsmål 3: Indeks - barnenodene

I en binærhaug har du en node med indeks 10. Hva er indeksen til de to barnenodene?

- ☐ 11 og 12
- ☐ 11 og 13
- ☐ 20 og 21

Spørsmål 4: Søke i tre

Trær har mange praktiske anvendelser. En av anvendelsene er å analysere lange strenger, hvor såkalte [suffikstrær](#) brukes til blant annet å finne likheter mellom strenger. I biologi er slike sammenligninger av strenger en viktig operasjon, ettersom man ønsker å kunne sammenligne DNA-/protein-sekvenser (strenger). Dette fordi DNA-/protein-sekvenser som ligner på hverandre som regel har lignende egenskaper. Ved å finne DNA-/protein-sekvenser som ligner på en ny ukjent sekvens, eller som ligner på en delsekvens av denne nye sekvensen, kan man estimere funksjonen og egenskapene til denne ukjente sekvensen.

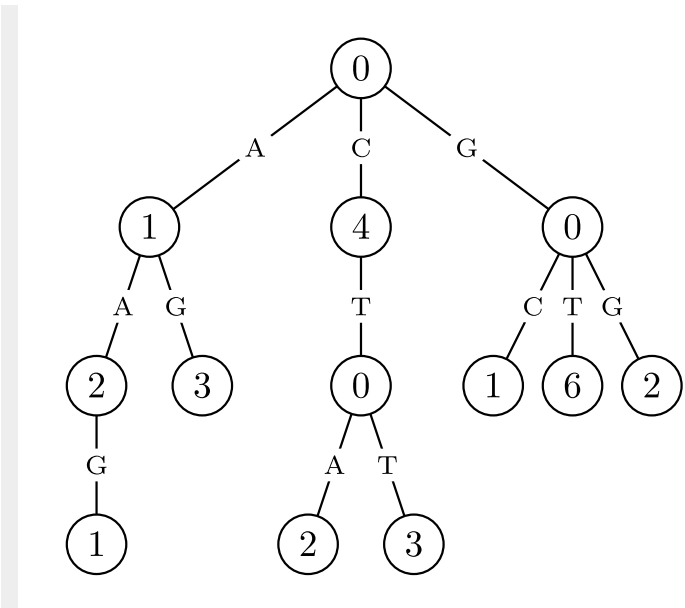
I de påfølgende oppgavene skal vi se på en enklere variant av dette problemet, hvor vi ikke tillater forskjeller mellom strengene. Vi er altså ute etter eksakte matcher mellom DNA-sekvenser.

I denne oppgaven skal du slå opp i et tre for å se om en viss DNA-sekvens er sett før. Treet består av **Node**-objekter, en type som er laget for deg.

Disse objektene har to attributter:

- children** er en oppslagstabell (*dictionary*) med alle nodebarna til noden. Den angir kobling mellom bokstav og node. For å gå til et barn kan man skrive `current_node = current_node.children["A"]`, gitt at `current_node` er den nåværende noden og at denne noden har et barn "A".
- count** er et heltall som holder styr på hvor mange DNA-sekvenser det finnes i databasen som er nøyaktig lik den korresponderende strengen til noden. Den korresponderende strengen er strengen som dannes av bokstavene langs stien fra rotnoden ned denne noden.

Treet kan se ut som følgende. Her ser vi at det finnes det én forekomst av DNA-sekvensen "AAG", fire forekomster av "C" og ingen forekomster av "CT".



Din oppgave er å implementere funksjonen `search_tree(root, dna)`. `root` er **Node**-objektet i roten av treet, mens `dna` er tekststrengen (altså DNA-sekvensen) du skal søke etter. Funksjonen skal returnere hvor mange forekomster det er i databasen av denne strengen.

Ønsker du teste programmet ditt lokalt, finnes det eksemplertester [her](#). Trenger du flere instanser å teste på lokalt, finnes det et større testsett som kan [lastes ned](#).

Hint 1

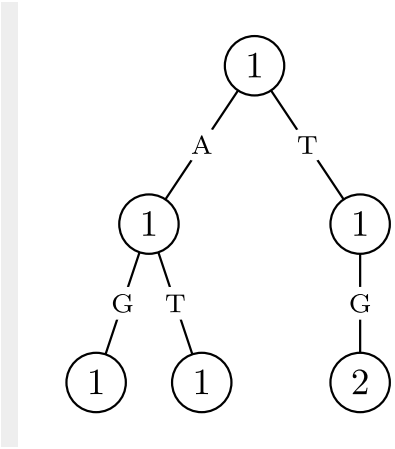
```
1 def search_tree(root, dna):
2     # Skriv koden din her
```

Spørsmål 5: Bygge tre

Du har nå implementert tresøk, men vi trenger også et tre å søke i.

I denne oppgaven skal lage funksjonen `build_tree(dna_sequences)` som bygger et søketre (bestående av `Node`-objekter). Input er en liste av strenger som treet skal bestå av. Funksjonen skal returnere roten til treet. Et tomt `Node`-objekt med `count = 0` og `children = {}` kan lages ved å kalle `Node()`.

Eksempel: Hvis du får inn `["A", "AG", "TG", "T", "TG", "", "AT"]` som liste med strenger, så skal algoritmen din lage det følgende treet:



Ønsker du teste programmet ditt lokalt, finnes det eksempeltester [her](#). Trenger du flere instanser å teste på lokalt, finnes det et større testsett som kan [lastes ned](#).

```
1 def build_tree(dna_sequences):
2     # Skriv koden din her
```

Spørsmål 6: Binærhaug kontra binært søketre

Anta at du bygger binære hauger og søketrær over minst tre elementer, alle unike. Hvilket utsagn er da korrekt?

- ☐ Noen binære søketrær er binære hauger.
- ☐ Alle binære hauger er binære søketrær.
- ☒ Ingen binære hauger er binære søketrær.
- ☐ Alle binære søketrær er binære hauger.
- ☐ Noen binære hauger er binære søketrær.

Spørsmål 7: De største elementene i en komplett binær min-haug

Denne oppgaven er basert på en eksamensoppgave.

I en komplett binær min-haug med n elementer, ligger de $\lceil n/2 \rceil$ største elementene i løvnodene?

Hint 1

- ☐ Nei, aldri
- ☐ Nei, ikke generelt
- ☒ Ja, alltid

Spørsmål 8: Sortert tabell kontra binærhaug

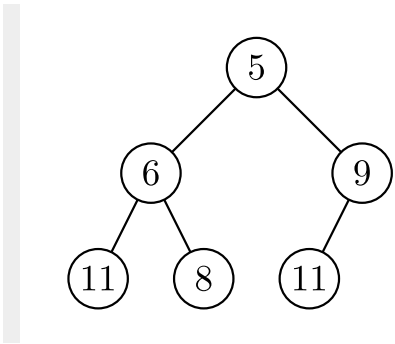
Representerer en sortert tabell med tall en binærhaug?

- ☐ Nei, aldri
- ☒ Nei, ikke generelt

☐ Ja, alltid

Spørsmål 9: Tabellrepresentasjon av binærhaug

Hvilken av tabellene under representerer følgende binærhaug?



- ☒ $\langle 5, 6, 9, 11, 8, 11 \rangle$
- ☐ $\langle 5, 6, 8, 9, 11, 11 \rangle$
- ☐ $\langle 11, 6, 8, 5, 9, 11 \rangle$
- ☐ $\langle 5, 6, 11, 8, 9, 11 \rangle$

Spørsmål 10: Heapsort

Hva er sant om **HEAPSORT**?

- ☐ Algoritmen er *in-place*.
- ☐ Det er en stabil sorteringsalgoritme.
- ☐ Algoritmen har kjøretid $O(n)$ om alle elementene har samme verdi.

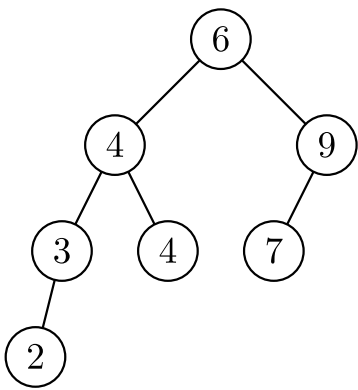
Spørsmål 11: Operasjoner på min-prioritetskø

Hvilke operasjoner er støttet av en min-prioritetskø?

- ☐ **SEARCH**: søker etter og returnerer et spesifikt element i prioritetskøen.
- ☐ **EXTRACT-MAX**: returnerer og sletter det største elementet i priortetskøen.
- ☐ **INCREASE-KEY**: øker verdien til et element i prioritetskøen.
- ☐ **MAXIMUM**: returnerer det største elementet i prioritetskøen.
- ☐ **MINIMUM**: returnerer det minste elementet i prioritetskøen.
- ☐ **EXTRACT-MIN**: returnerer og sletter det minste elementet i priortetskøen.
- ☐ **INSERT**: legger til et element i prioritetskøen.
- ☐ **DECREASE-KEY**: senker verdien til et element i prioritetskøen.

Spørsmål 12: Binærtre - innsetting

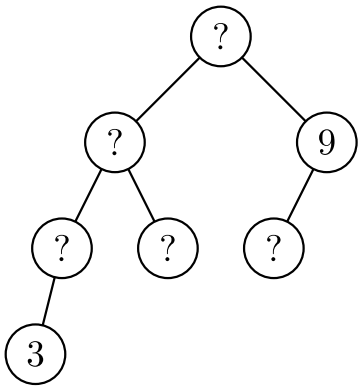
Du setter tallene i en tabell **A** inn i et binært søketre **T** i den rekkefølgen tallene står i **A**. Hvis du setter inn tallene ved hjelp av **TREE-INSERT**, og **T** ser ut som på figuren under, hvilke av disse alternativene kunne ha vært **A**?



- ☐ $\langle 6, 4, 4, 3, 7, 9, 2 \rangle$
- ☐ $\langle 6, 9, 4, 3, 2, 7, 4 \rangle$
- ☐ $\langle 6, 4, 3, 9, 2, 7, 4 \rangle$
- ☐ $\langle 6, 9, 3, 7, 2, 4, 4 \rangle$
- ☐ $\langle 6, 4, 9, 3, 4, 7, 2 \rangle$
- ☐ $\langle 6, 4, 4, 9, 2, 3, 7 \rangle$

Spørsmål 13: Finn nodeverdier

Treet under er et binært søketre hvor noen av elementene er byttet ut med spørsmålstegn («?»). Hvis man erstatter spørsmålstegnene med tallverdier, slik at man ender opp med et gyldig binært søketre, hva er den største og minste mulige summen av nøkler i det binære søketreet det er mulig å oppnå?



- ☐ Minste er 31 og største er 51
- ☐ Minste er 27 og største er 57
- ☐ Minste er 38 og største er 48
- ☐ Minste og største er begge 42

Spørsmål 14: Binærtre - sortering

Betrakt følgende sorteringsalgoritme:

Først setter du alle tallene inn i et binært søketre T ved hjelp av algoritmene i pensum, og deretter kjører du $\text{INORDER-TREE-WALK}(T.\text{root})$.

Hva blir kjøretiden til denne algoritmen i beste og verste tilfelle, samt den gjennomsnittlige kjøretiden til algoritmen?

- ☐ Kjøretiden er $\Theta(n \lg n)$ i alle tilfeller.
- ☐ Kjøretiden er $\Theta(n)$ i beste tilfelle, den gjennomsnittlige kjøretiden er $\Theta(n \lg n)$ og kjøretiden er $\Theta(n^2)$ i verste tilfelle.
- ☐ Både kjøretiden i beste tilfelle og den gjennomsnittlige kjøretiden er $\Theta(n \lg n)$, mens kjøretiden er $\Theta(n^2)$ i verste tilfelle.
- ☐ Kjøretiden er $\Theta(n \lg n)$ i beste tilfelle, men både den gjennomsnittlige kjøretiden og kjøretiden i verste tilfelle er $\Theta(n^2)$
- ☐ Kjøretiden til algoritmen er $\Theta(n^2)$ i alle tilfeller.

Spørsmål 15: Binærtre - sortering

Hva er sant om sorteringsalgoritmen i oppgaven over?

- ☐ Hvis alle elementene i tabellen du skal sortere er like, vil algoritmen ha kjøretid $O(n)$ siden tabellen allerede er sortert.
- ☐ Sorteringsalgoritmen er asymptotisk optimal i verste tilfelle for sammenligningsbasert sortering.
- ☐ Sorteringsalgoritmen er stabil dersom du setter inn tallene i søketreet i samme rekkefølge som de var i den opprinnelige tabellen.

Spørsmål 16: Binærtre - verste kjøretid

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Med algoritmene i pensum kan man bygge binære hauger i lineær tid, men det er ingen algoritmer i pensum som forteller hvordan man kan bygge et binært søketre i lineær tid i verste tilfelle.

Finn en slik algoritme, eller eventuelt bevis hvorfor en slik algoritme ikke kan eksistere.

Hint 1

Hint 2

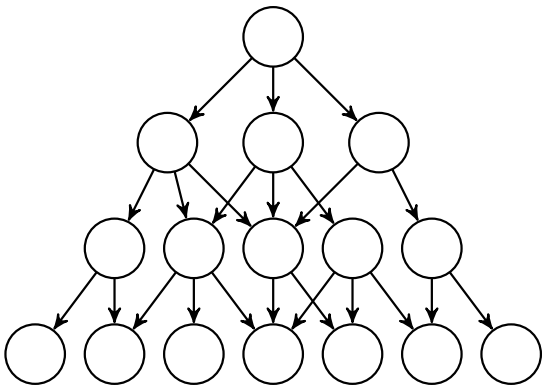
Spørsmål 17: Ikke et tre

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Denne oppgaven er et frempek i pensum og kan oppleves som vanskelig. Om du ikke får den til, fortvil ikke.

En egenskap ved trær er at det kun er én sti fra en rot til hver av etterkommernodene. Vi går nå over til en lignende datastruktur der en node kan ha én eller flere foreldre. Hvor mange ulike stier er det da fra rota til hver av etterkommernodene, dersom alle stiene kun skal gå nedover i denne treliknende datastrukturen? Er det mulig å finne ut dette i lineær tid i verste tilfelle, som funksjon av antall kanter?

En visualisering av hva slags datastruktur vi snakker om:



Hint 1

Hint 2

Hint 3

Spørsmål 18: Strengmatching

I stedet for å finne eksakte matcher mellom hele strenger, ønsker vi nå å finne ut hvor mange ganger korte DNA-segmenter opptrer i en lang DNA-sekvens.

I denne oppgaven skal du skrive en funksjon `string_match(dna, segments)`. Her er `dna` DNA-sekvensen du skal søke i (representert som en tekststreng), og `segments` en liste av DNA-segmenter du skal finne (representert som en liste med tekststrenger). Funksjonen skal returnere antall ganger DNA-segmentene forekommer i DNA-sekvensen. Du kan anta at ingen av DNA-segmentene i `segments` er tomme (en tekststreng er tom hvis den ikke inneholder noen symboler).

Eksempel: Om man får inn `dna = "ACTTACTGG"` og `segments = ["A", "ACT", "GG"]` skal funksjonen returnere `5`, siden `"A"` og `"ACT"` begge forekommer to ganger, mens `"GG"` forekommer kun én gang.

Du får her tilgang på `search_tree(root, dna)` og `build_tree(dna_sequences)`. Disse er implementert av oss som definert i de tidligere programmeringsoppgavene. Det er frivillig om man vil benytte seg av disse.

Hvis lengden på `dna` er n , antallet DNA-segmenter er k og det lengste av DNA-segmentene er d , må koden din ha en kjøretid på $O(d(n + k))$ for å bli godkjent.

Ønsker du teste programmet ditt lokalt, finnes det eksempltester [her](#).

Denne oppgaven teller mot poengtavlen til denne øvingen. For å kunne havne på poengtavlen må du løse oppgaven og sette variabelen `highscore` til `True` i koden din.

Rangeringen din er basert på hvor lang tid det tar å finne antall matcher på flere input av DNA-sekvenser og DNA-segmenter.

```
1 def string_match(dna, segments):
2     # Skriv koden din her
3
4
5 # Sett 'highscore' til True hvis du vil vises på poengtavlen.
6 # For mer info se 'https://algdat.idi.ntnu.no/ovinger.html#poengtavle'
7 # Merk: Kjøring for poengtavlen tar betraktelig lengre tid.
8 highscore = False
```

Spørsmål 19: Høyde - binærtre

Hvilke beskrivelser kan vi gi av høyden til et binært søketre med n noder uten å vite noe mer om det binære søketreet?

- ☐ $\Omega(n)$
- ☐ $O(\lg n)$
- ☐ $\Theta(n)$
- ☐ $\Theta(\lg n)$
- ☐ $O(n)$
- ☐ $\Omega(\lg n)$

Spørsmål 20: Høyde - binærhaug

Hvilke beskrivelser kan vi gi av høyden til en binærhaug med $n!$ elementer uten å vite noe mer om binærhaugen?

- ☐ $\Theta(n)$
- ☐ $O(n)$
- ☐ $\Theta(\lg n)$
- ☐ $O(\lg(n!))$
- ☐ $\Omega(\lg n)$
- ☐ $\Theta(n \lg n)$
- ☐ $\Omega(\lg(n!))$

Spørsmål 21: Forhold i binærtre

Denne oppgaven er vanskelig. Fortvil ikke om du ikke får den til.

Hvilke av følgende påstander stemmer for alle binærtrær med n noder?

- ☐ Hvis antall løvnoder er $O(1)$ er høyden på treet $\Theta(n)$.
- ☐ Hvis antall interne noder med kun én barnenode er $\Theta(n)$ er høyden på treet $\Theta(n)$.
- ☐ Hvis det er flere løvnoder enn det er interne noder i et binærtre er høyden på treet $\Theta(\lg n)$.
- ☐ Hvis antall interne noder med to barn er k og antall løvnoder er ℓ , så er $k + 1 = \ell$.
- ☐ Et komplett binærtre har høyde $\Theta(\lg n)$.
- ☐ Et komplett binærtre (alle interne noder har to barnenoder og alle løvnoder er på samme nivå) har $\lfloor n/2 \rfloor$ løvnoder.
- ☐ Et komplett binærtre har $\lfloor n/2 \rfloor$ interne noder.

Spørsmål 22: Ikke-binære trær

Du synes høyden på binære trær er for stor, så du ser om du kan minske høyden ved å la noder ha flere enn to barnenoder. Hva er den minste høyden som er mulig på et rotfast tre med $n \gg 1$ noder der hver node kan ha opptil 2048 barnenoder?

- ☐ $\Theta(\lg n)$
- ☐ $\Theta(\lg \lg n)$
- ☐ $\Theta(1)$

Spørsmål 23: Ikke-binære trær

Du har flere ideer for å få ned høyden på treet ditt. En ny ide er å gjøre antall barn en node kan ha til en funksjon av antallet etterkommernoder. Hvis en node i et rotfast tre som har k etterkommernoder kan ha $\lfloor \sqrt{k} \rfloor$ barnenoder, og treet har $n \gg 1$ noder, hva blir da den minste mulige høyden på treet?

- ☐ $\Theta(\lg \lg n)$
- ☐ $\Theta(\lg n)$
- ☐ $\Theta(1)$

Spørsmål 24: Indeksering

Du har en tabell med informasjon om ulike produkter, der alle produktene har et unikt identifikasjonsnummer som man bruker til å søke seg fram til informasjonen om produktet. I tillegg kan det komme nye produkter etterhvert, samt at gamle produkter kan fjernes.

Du skal finne en algoritme/datastruktur som støtter dette ved at man kan slå opp på indentifikasjonnummer, legge til nye varer med identifikasjonsnummer, og slette varer med bestemte identifikasjonsnummer. Du har satt opp fem ulike alternativ:

1. Bruke et binært søketre, som beskrevet i pensum, med identifikasjonsnummer som søkenøkkel.
2. Bruke et balansert binært søketre med identifikasjonsnummer som søkenøkkel.
3. Bruke en hashtabell (som beskrevet i kapittel 11.2 i CLRS), med indentifikasjonsnummer som søkenøkkel.
4. Bruke en dynamisk tabell, der du legger til nye varer bakerst.
5. Bruke en dynamisk tabell som i 4, men nå sorterer du tabellen etter hver innsetting.

Du vet ingenting om hvordan identifikasjonsnumrene er fordelt. Hvilke av påstandene under stemmer?

- ☐ Tre av alternativene kan oppnå $o(n)$ kjøretid i verste tilfelle for søk.
- ☐ Alternativ 1, 2, 3 og 5 utfører alle søk med $O(\lg n)$ som gjennomsnittlig kjøretid.

- ☐ Hvis du bruker alternativ 5 så vil hver innsetting garantert ha kjøretid $\Omega(n \lg n)$ i verste tilfelle dersom man bruker en sammenligningsbasert sorteringsalgoritme, siden det er verste tilfelle for sammenligningsbaserte sorteringsalgoritmer.
- ☐ Alternativ 1, 3 og 4 har alle kjøretid $O(1)$ i beste tilfelle ved innsetting.

Spørsmål 25: Turnering

Denne oppgaven er vanskelig. Fortvil ikke om du ikke får den til.

Du organiserer en turnering med n deltagere. Du har en tabell over poengsummen for hver av deltagerene, men du ønsker også å vite hvem som leder (altså er på foreløpig første plass) til enhver tid. Deltagerene kan kun øke sin poengsum, man går aldri ned i poeng, og alle starter med 0 poeng. La k være antall ganger en deltager oppdaterer poengsummen sin.

Du har funnet fire ulike løsninger for å holde orden på hvem som leder:

- Når du oppdaterer poengsummen til en deltager kjører du **MERGE-SORT** på tabellen over poengsummer.
- Når du oppdaterer poengsummen til en deltager kjører du **INSERTION-SORT** på tabellen over poengsummer.
- Du har i en maks-haug en node for hver deltager, og hver gang du oppdaterer poengsummen til en deltager, kjører du **HEAP-INCREASE-KEY** på noden til deltageren.
- Du har en maks-haug, og hver gang du oppdaterer en deltagers poengsum setter du inn en ny node med den nye poengsummen til deltageren.

Hvilke av påstandene under stemmer?

- ☐ **MERGE-SORT** passer bedre enn **INSERTION-SORT** i denne situasjonen, siden **MERGE-SORT** har bedre gjennomsnittlig kjøretid.
- ☐ Alle algoritmene har kjøretid $O(n)$ for en oppdatering hvis $k = O(n)$.
- ☐ Tre av de fire algoritmene har kjøretid $O(1)$ for en oppdatering i beste tilfelle.
- ☐ Alle algoritmene bruker $O(n)$ minne uansett hva k er.
- ☐ Hvis $k = \Theta(n^3)$ bruker algoritme 3 og 4 like lang tid (asymptotisk, i verste tilfelle) på en oppdatering.
- ☐ Algoritme 3 er like rask som algoritme 4 på en oppdatering (asymptotisk, i verste tilfelle) uansett hva k er.
- ☐ Kjøretiden til en oppdatering med algoritme 4 er $\Theta(\lg(n^k))$ i verste tilfelle.
- ☐ Algoritme 4 bruker like lang tid (asymptotisk, i verste tilfelle) på en oppdatering enten $k = n!$ eller $k = n^n$.

Spørsmål 26: Repeterte delstrenger

Denne oppgaven er vanskelig. Fortvil ikke om du ikke får den til.

En annen viktig anvendelse av trær i biologi er å finne delstrenger i DNA-/protein-sekvenser som opptrer flere ganger i samme sekvens.

I denne oppgaven skal du lage en funksjon `longest_repeated_substring(dna, k)`. Her er `dna` DNA-sekvensen du skal finne repeterende, muligens delvis overlappende, delstrenger i. Funksjonen skal returnere den lengste delstrengen i `dna` som opptrer minst `k` ganger. Om det ikke er noen delstreng som opptrer `k` ganger i `dna`, skal funksjonen returnere `None`. Om det er flere forskjellige delstrenger som repeteres minst `k` ganger og som har samme maksimale lengde, kan du selv velge hvilken du vil returnere. Du kan anta at `dna` er en ikke-tom streng og at den kun inneholder kombinasjoner av de fire bokstavene `ACGT` og at $k \geq 2$.

Eksempel: Om man får inn `dna = "ACTACTAGC"` og `k = 2` skal funksjonen returnere `"ACTA"`, siden `"ACTA"` forekommer to ganger i `dna` (`dna[0:4]` og `dna[3:7]`). Om `k = 3` skal man derimot returner `"A"` eller `"C"`, mens for `k = 4` skal man returnere `None`.

Hvis lengden på `dna` er n , må koden din ha en kjøretid på $O(n^2)$ for å bli godkjent. *(Det mulig å oppnå en mye bedre asymptotisk kjøretid enn dette.)*

Ønsker du teste programmet ditt lokalt, finnes det eksempltester [her](#).

Denne oppgaven teller mot poengtavlen til denne øvingen. For å kunne havne på poengtavlen må du løse oppgaven og sette variabelen `highscore` til `True` i koden din.

Rangeringen din er basert på hvor lang tid det tar å finne lengste repeterte delstreng i flere input av DNA-sekvenser.

Hint 1

```
1 def longest_repeated_substring(dna, k):
2     # Skriv koden din her
3
4
5 # Sett 'highscore' til True hvis du vil vises på poengtavlen.
6 # For mer info se 'https://algdat.idi.ntnu.no/ovinger.html#poengtavle'
7 # Merk: Kjøring for poengtavlen tar betraktelig lengre tid.
8 highscore = False
```

Spørsmål 27: Anbefalte ekstraoppgaver

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Her følger en liste over anbefalte ekstraoppgaver, som kan være nyttig å se på om du ønsker litt ekstra trening eller som repetisjon mot eksamen. Oppgavene er hentet fra [gamle eksamensoppgaver](#) og *Introduction to Algorithms* av Cormen mfl. (CLRS), og er sortert basert på kilde.

- [2026S](#) - 15
- [2025H](#) - 1
- [2025S](#) - 4
- [2024H](#) - 4, 5
- [2023H](#) - 1e
- [2023S](#) - 13
- [2022H](#) - 8, 11
- [2022S](#) - 4
- [2021H](#) - 4
- [2020H](#) - 4
- [2020S](#) - 1
- [2019H](#) - 5, 9
- [2019S](#) - 8
- [2018H](#) - 9
- [2018S](#) - 13
- [2017S](#) - 3
- [2016H](#) - 4, 6
- [2016S](#) - 9, 10
- [2015H](#) - 14, 15
- [2015S](#) - 11
- [2014S](#) - 6
- **CLRS** - 6.1-1, 6.1-4, 6.1-5, 6.1-8
- **CLRS** - 6.2-2, 6.2-6, 6.2-7
- **CLRS** - 6.3-2, 6.3-4
- **CLRS** - 6.4-5 (vanskelig)
- **CLRS** - 6.5-1, 6.5-2, 6.5-7, 6.5-11
- **CLRS** - 6-3
- **CLRS** - 12.1-5
- **CLRS** - 12.2-4, 12.2-5, 12.2-6, 12.2-7, 12.2-8, 12.2-9
- **CLRS** - 12-1, 12-3

Send inn